MyCut - Smart Contract Security Review

Portfolio: solidity-security-casebook
Challenge: CodeHawks AI First Flight #8 - MyCut #8 **Date:** Oct 31st, 2026 - Nov 2nd, 2026

Table of contents

- **Findings Summary**
- **Matched Findings**
- **Additional Findings**

About the Project

MyCut distributes contest rewards through individual Pot contracts. The owner creates and funds each Pot. Players have 90 days to claim their assigned rewards. After that, the manager takes 10% of the remaining pool, rounded down, and the rest should be shared equally among players who claimed in time.

Actors

- **Owner / Admin (trusted):** creates, funds and closes contests.
- **Player:** claims an assigned reward.

Scope and compatibility

- Contracts: `src/ContestManager.sol` and `src/Pot.sol`.
- Chains: EVM-equivalent chains only.
- Tokens: standard ERC-20 tokens only.
- The original README lists no known issues.

Project details are adapted from the original repository README. Its 2024 dates describe the original First Flight #23, not this review.

Review Details

- Source repository
- AI challenge
- Reviewed commit: `819134663950999ca5ab29a91eeceddb80274743`.
- Tools: manual review, Foundry 1.5.1 and Solidity 0.8.20.
- Verification: all 11 reviewed PoC tests passed against the original contracts. No fuzz or invariant-testing campaign was performed.

This is a personal learning review prepared with AI assistance. It is not an official Cyfrin audit. The PoCs below are concise scenario summaries; full executable tests and repeated helper code are omitted.

Findings Summary

The AI judge matched three submitted topics: two High issues and one Low issue. The H/L labels below follow those displayed matches. A-IDs identify additional observations without an accepted severity.

ID	Finding	AI result
H-01	Manager fees are locked in `ContestManager`	Matched - **High**
H-02	Closing divides by all players instead of claimants	Matched - **High**
L-01	Promised rewards can exceed `totalRewards`	Matched - **Low**
A-01	Claims remain possible after the deadline and closing	Not matched
A-02	Closing can repeat payments	Not matched
A-03	A false transfer result consumes a claim without payment	Not matched
A-04	Repeated funding can leave extra tokens locked	Not matched
A-05	One reverting claimant transfer blocks closing	Not matched
A-06	Extra reward entries are silently ignored	Not matched

“Not matched” records the AI evaluation, not a conclusion that the tested behavior cannot occur. Evaluation reasons below come from the supplied feedback. Review limitations are stated separately.

Matched Findings

High Risk Findings

H-01. Manager cuts are sent to `ContestManager`, which has no function to withdraw tokens, so it is locked permanently

Root + Impact

`Pot::closePot` sends the manager's cut to `msg.sender`, which is the `ContestManager` contract (the Pot's owner), not the human owner. `ContestManager` has no function that can move ERC-20 tokens out, so the manager's cut of every closed contest is permanently locked in `ContestManager`. This is independent of the divisor bug in `closePot`: fixing that bug does not change where the cut goes.

Description

The README states that after 90 days the manager takes a cut of the remaining pool. The cut is transferred like this:

```
// Pot.sol
contract Pot is Ownable(msg.sender) { ... } // owner = whoever deploys the
Pot

function closePot() external onlyOwner {
    ...
   i_token.transfer(msg.sender, managerCut); // msg.sender = ContestManager
}
```

The `Pot` is deployed by `ContestManager`, and it is also `ContestManager` that calls `closePot`:

```
// ContestManager.sol
function createContest(...) public onlyOwner returns (address) {
    Pot pot = new Pot(players, rewards, token, totalRewards); // Pot owner
= ContestManager
    ...
}

function _closeContest(address contest) internal {
    Pot pot = Pot(contest);
    pot.closePot(); // msg.sender inside closePot = ContestManager
}
```

So the tokens end up on the `ContestManager` address. Its external functions are:

```
┌-------------------------------------------------------+------------┐
| Method                                                | Identifier |
+=======================================================================+
| closeContest(address)                                 | b24eaac8   |
|-------------------------------------------------------+------------|
| contestToTotalRewards(address)                        | 1d01381c   |
|-------------------------------------------------------+------------|
| contests(uint256)                                     | 9d7ed738   |
|-------------------------------------------------------+------------|
| createContest(address[],uint256[],address,uint256)    | de9b18c7   |
|-------------------------------------------------------+------------|
| fundContest(uint256)                                  | b6bfbe16   |
|-------------------------------------------------------+------------|
| getContestRemainingRewards(address)                   | 2426551e   |
|-------------------------------------------------------+------------|
| getContestTotalRewards(address)                       | e172d5ba   |
|-------------------------------------------------------+------------|
| getContests()                                         | bfaba76a   |
|-------------------------------------------------------+------------|
| owner()                                               | 8da5cb5b   |
|-------------------------------------------------------+------------|
| renounceOwnership()                                   | 715018a6   |
|-------------------------------------------------------+------------|
```

```
| transferOwnership(address)                          | f2fde38b   |
└-----------------------------------------------------+------------┘
```

None of them transfer ERC-20 tokens out. The inherited `Ownable` functions (`transferOwnership`, `renounceOwnership`) do not touch token balances.

Risk

Likelihood: High

Impact: High

Proof of Concept

Add the following functions inside the existing `TestMyCut` contract in `test/TestMyCut.t.sol`. The test uses its existing setup, imports, and `mintAndApproveTokens` modifier.

```
address player3 = makeAddr("player3");

function _createFundedPot(
    uint256 r1,
    uint256 r2,
    uint256 r3,
    uint256 total
) internal returns (Pot) {
    address[] memory p = new address[](3);
    p[0] = player1;
    p[1] = player2;
    p[2] = player3;

    uint256[] memory r = new uint256[](3);
    r[0] = r1;
    r[1] = r2;
    r[2] = r3;

    vm.startPrank(user);
    contest = ContestManager(conMan).createContest(
        p,
        r,
        IERC20(ERC20Mock(weth)),
        total
    );
    ContestManager(conMan).fundContest(0);
    vm.stopPrank();

    return Pot(contest);
}

function test_PoC_managerCut_stuckInContestManager() public
mintAndApproveTokens {
    Pot pot = _createFundedPot(10, 20, 30, 60);
```

```
    vm.prank(player1);
    pot.claimCut();

    uint256 ownerBeforeClose = weth.balanceOf(user);
    vm.warp(block.timestamp + 91 days);
    vm.prank(user);
    ContestManager(conMan).closeContest(address(pot));

    assertEq(weth.balanceOf(conMan), 5);
    assertEq(weth.balanceOf(user), ownerBeforeClose);
}
```

Run from the repository root:

```
forge test --mt test_PoC_managerCut_stuckInContestManager -vv
```

Output:

```
[PASS] test_PoC_managerCut_stuckInContestManager() (gas: 1193435)
Logs:
  User Address:  0x6CA6d1e2D5347Bfab1d91e883F1915560e09129D
  Contest Manager Address 1:  0x7BD1119CEC127eeCDBa5DCA7d1Bd59986f6d7353
  Minting tokens to:  0x6CA6d1e2D5347Bfab1d91e883F1915560e09129D
  Approved tokens to:  0x7BD1119CEC127eeCDBa5DCA7d1Bd59986f6d7353
```

Recommended Mitigation

Give the manager a way to receive the cut. Either add a withdrawal function to `ContestManager`:

```
+ using SafeERC20 for IERC20;   // import SafeERC20 as the repo imports
OpenZeppelin
+
+  function withdrawTokens(IERC20 token, address to, uint256 amount)
external onlyOwner {
+     token.safeTransfer(to, amount);
+ }
```

or send the cut directly to a manager address instead of `msg.sender` in `Pot::closePot`.

H-02. `Pot::closePot` divides by `i_players.length` instead of `claimants.length`, resulting in incomplete reward distribution

Root + Impact

`Pot::closePot` divides the claimants' share by the number of all registered players (`i_players.length`) instead of the number of players who claimed in time (`claimants.length`). Whenever at least one player does not claim, claimants receive only `claimants.length / i_players.length` of the amount the protocol promises them, and the rest stays undistributed in the Pot.

Description

According to the protocol description, rewards remaining after the 90-day claim period should be distributed equally among players who claimed in time, after deducting the manager's share.

However, `Pot::closePot` divides the distributable amount by the total number of registered players:

```
uint256 claimantCut = (remainingRewards - managerCut) / i_players.length;
for (uint256 i = 0; i < claimants.length; i++) {
    _transferReward(claimants[i], claimantCut);
}
```

The resulting amount is transferred only to addresses in claimants. Therefore, when some registered players do not claim, eligible claimants receive less than their intended share.

For example, a Pot contains **60 tokens** allocated to 3 players with rewards of **10, 20, and 30**. Only the first player claims, leaving **50 tokens**.

At closure, the manager receives 5, leaving 45 for the sole claimant. Instead of transferring all 45, the contract transfers **45 / 3 = 15**, leaving **30 tokens** undistributed in the Pot.

Risk

Likelihood: High

It happens on every close the pot in which at least one player did not claim within 90 days. This is the main scenario the protocol is designed for, and no attacker or special condition is required.

Impact: High

Eligible claimants receive less than the amount specified by the protocol. In the demonstrated scenario, the sole claimant receives a closing payment of 15 instead of 45, leaving 30 tokens undistributed.

The PoC demonstrates incomplete distribution during closure, rather than permanent freezing of the remaining funds.

Proof of Concept

Add the following functions inside the existing `TestMyCut` contract in `test/TestMyCut.t.sol`. The test uses its existing setup, imports, and mintAndApproveTokens modifier.

```
address player3 = makeAddr("player3");

 function _createFundedPot(
```

```
        uint256 r1,
        uint256 r2,
        uint256 r3,
        uint256 total
    ) internal returns (Pot) {
        address[] memory p = new address[](3);
        p[0] = player1;
        p[1] = player2;
        p[2] = player3;

        uint256[] memory r = new uint256[](3);
        r[0] = r1;
        r[1] = r2;
        r[2] = r3;

        vm.startPrank(user);
        contest = ContestManager(conMan).createContest(
            p,
            r,
            IERC20(ERC20Mock(weth)),
            total
        );

        ContestManager(conMan).fundContest(0);

        vm.stopPrank();
        return Pot(contest);
    }

    function test_PoC_closePotDivisionByAllPlayersBug() public
    mintAndApproveTokens {
        Pot pot = _createFundedPot(10, 20, 30, 60);

        vm.prank(player1);
        pot.claimCut();

        vm.warp(block.timestamp + 91 days);

        vm.prank(user);
        ContestManager(conMan).closeContest(address(pot));

        console.log("player1 balance:", weth.balanceOf(player1));
        console.log("conMan balance :", weth.balanceOf(conMan));
        console.log("pot balance    :", weth.balanceOf(address(pot)));

        assertEq(weth.balanceOf(player1), 10 + 15);
        assertEq(weth.balanceOf(conMan), 5);
        assertEq(weth.balanceOf(address(pot)), 30);
        }
```

Run from the repository root:

```
forge test --mt test_PoC_closePotDivisionByAllPlayersBug -vv
```

Output:

```
[PASS] test_PoC_closePotDivisionByAllPlayersBug() (gas: 1202067)
Logs:
 ...
  player1 balance: 25
  conMan balance : 5
  pot balance    : 30
```

Recommended Mitigation

Divide the distributable rewards by the number of claimants:

```
- uint256 claimantCut = (remainingRewards - managerCut) / i_players.length;
+ uint256 claimantCut = (remainingRewards - managerCut) / claimants.length;
```

Handle claimants.length == 0 before performing the division, and define how any rounding remainder should be distributed.

Low Risk Findings

L-01. Pot is created without checking that the rewards add up to `totalRewards`, so some players cannot claim their reward

Root + Impact

`ContestManager::createContest` and the `Pot` constructor never check that `sum(rewards) == totalRewards`. `fundContest` sends only `totalRewards` tokens to the Pot. If the rewards add up to more than `totalRewards`, the Pot does not hold enough tokens for all promised rewards. The player who claims last cannot claim: `claimCut` reverts with an arithmetic underflow, and the player's reward can never be paid.

Description

The constructor stores the rewards and the total separately and never compares them:

```
constructor(
        address[] memory players,
        uint256[] memory rewards,
        IERC20 token,
```

```
        uint256 totalRewards
    ) {
        ...
        remainingRewards = totalRewards;
        ...

       // no check that sum(rewards) == totalRewards
        for (uint256 i = 0; i < i_players.length; i++) {
            playersToRewards[i_players[i]] = i_rewards[i];
        }
    }
```

`Pot::claimCut` subtracts each reward from `Pot::remainingRewards`, so when the promised rewards are more than the total, the last claim goes below zero and reverts:

```
remainingRewards -= reward; // underflow when the rewards add up to more
than totalRewards
```

Example: rewards 10 / 20 / 40 (70 promised), `totalRewards` = 60. After player1 and player2 claim, 30 tokens remain, but player3 is owed 40. player3's `claimCut` reverts and player3 gets nothing.

Risk

Likelihood: Low

The owner must pass numbers that do not add up (for example, by a typing mistake). No attacker is involved. Nothing in the code or in `fundContest` warns about it, so the mistake is not noticed before players try to claim.

Impact: Medium

One or more players cannot claim the reward they were promised. In the PoC, player3 is owed 40 and gets 0. The protocol's own documentation says players are "authorized" to claim these amounts, so this breaks the promise. Only the money of this one contest is affected.

Proof of Concept

Add the following functions inside the existing `TestMyCut` contract in `test/TestMyCut.t.sol`.

```
address player3 = makeAddr("player3");

 function _createFundedPot(
    uint256 r1,
    uint256 r2,
    uint256 r3,
    uint256 total
    ) internal returns (Pot) {
    address[] memory p = new address[](3);
```

```
        p[0] = player1;
        p[1] = player2;
        p[2] = player3;

        uint256[] memory r = new uint256[](3);
        r[0] = r1;
        r[1] = r2;
        r[2] = r3;

        vm.startPrank(user);
        contest = ContestManager(conMan).createContest(
        p,
        r,
        IERC20(ERC20Mock(weth)),
        total
        );

        ContestManager(conMan).fundContest(0);

        vm.stopPrank();

        return Pot(contest);
    }

    function test_PoC_rewardsSumExceedsTotal() public mintAndApproveTokens {
            Pot pot = _createFundedPot(10, 20, 40, 60); // 70 promised, 60
    funded

            vm.prank(player1);
            pot.claimCut(); // remainingRewards = 50

            vm.prank(player2);
            pot.claimCut(); // remainingRewards = 30

            vm.prank(player3);
            vm.expectRevert(
                abi.encodeWithSignature("Panic(uint256)", uint256(0x11))
            ); // remainingRewards (30) -= 40 underflows
            pot.claimCut();

            assertEq(pot.checkCut(player3), 40);
            assertEq(weth.balanceOf(player3), 0);
            assertEq(weth.balanceOf(address(pot)), 30);

            uint256 remainingRewards = ContestManager(conMan)
                .getContestRemainingRewards(address(contest));

            console.log("player3 is owed:", pot.checkCut(player3));
            console.log("player3 received:", weth.balanceOf(player3));
            console.log("tokens left in the Pot:",
    weth.balanceOf(address(pot)));
            console.log("Remaining rewards", remainingRewards);
```

```
        assertEq(remainingRewards, 30);
    }
```

Run from the repository root:

```
forge test --mt test_PoC_rewardsSumExceedsTotal -vv
```

Output:

```
[PASS] test_PoC_rewardsSumExceedsTotal() (gas: 1184922)
Logs:
  ...
  player3 is owed: 40
  player3 received: 0
  tokens left in the Pot: 30
  Remaining rewards 30
```

Recommended Mitigation

The Pot must never be created with rewards that do not add up to `totalRewards`. If this is guaranteed at creation, the Pot always holds exactly the amount that is promised (because `fundContest` sends `totalRewards`), and every `claimCut` call can be paid: `remainingRewards` and the sum of the unclaimed rewards decrease by the same amount, so the subtraction can never underflow.

The simplest place for this check is `ContestManager::createContest`, before the Pot is deployed. A wrong input then fails early, before any Pot is created:

```
+ error ContestManager__RewardsMismatch();

  function createContest(
      address[] memory players,
      uint256[] memory rewards,
      IERC20 token,
      uint256 totalRewards
  ) public onlyOwner returns (address) {
+     uint256 sum;
+     for (uint256 i = 0; i < rewards.length; i++) {
+         sum += rewards[i];
+     }
+     if (sum != totalRewards) revert ContestManager__RewardsMismatch();
      // Create a new Pot contract
      ...
  }
```

The check uses `!=` and not `>`, so it also covers the opposite case, `sum(rewards) < totalRewards`. There, the extra tokens would stay in the Pot and belong to nobody.

The same check can also be added in the `Pot` constructor, so the invariant holds for every way a Pot can be created.

Additional Observations

[A-01] Claims remain possible after the deadline and closing

Root + Impact

`Pot::closePot` checks that 90 days have passed, but `Pot::claimCut` never checks the deadline. A player who missed the 90 days can still claim their reward, and then also receives a share of the final distribution. This takes money away from the players who claimed in time and from the manager.

Description

The README says players have 90 days to claim. After that, the manager takes a cut and the rest goes to the players who claimed in time:

> allowing authorized claimants 90 days to claim before the manager takes a cut of the remaining pool and the remainder is distributed equally to those who claimed in time

`closePot` checks the 90 days, but `claimCut` does not:

```
function claimCut() public {
    address player = msg.sender;
    uint256 reward = playersToRewards[player];
    if (reward <= 0) {
        revert Pot__RewardNotFound();
    }
    playersToRewards[player] = 0;
    remainingRewards -= reward;
    claimants.push(player);
    _transferReward(player, reward);
}

function closePot() external onlyOwner {
    if (block.timestamp - i_deployedAt < 90 days) {
        revert Pot__StillOpenForClaim();
    }
    ...
}
```

So after day 90, and before the owner calls `closeContest`, any player can still claim. The late player is also added to `claimants`, so `closePot` pays them the same closing share as the players who claimed in time.

Risk

Likelihood: High

- Any player who missed the deadline can do this. No attacker and no special setup are needed.
- It only needs the owner not to have closed the contest yet, which is the normal situation after day 90.

Impact: Medium

- The late player takes their reward after the deadline. By the README, this reward should stay in the pool for the players who claimed in time.
- The late player is also counted as a claimant, so they get the same closing payment as the on-time claimants.
- As a result, the on-time claimants and the manager receive less than the protocol promises.
- Only the money inside this one contest is affected. No funds are lost from outside it.

Proof of Concept

Add the following functions inside the existing `TestMyCut` contract in `test/TestMyCut.t.sol`.

```
address player3 = makeAddr("player3");

 function _createFundedPot(
    uint256 r1,
    uint256 r2,
    uint256 r3,
    uint256 total
    ) internal returns (Pot) {
    address[] memory p = new address[](3);
    p[0] = player1;
    p[1] = player2;
    p[2] = player3;

    uint256[] memory r = new uint256[](3);
    r[0] = r1;
    r[1] = r2;
    r[2] = r3;

    vm.startPrank(user);
    contest = ContestManager(conMan).createContest(
    p,
    r,
    IERC20(ERC20Mock(weth)),
    total
    );

    ContestManager(conMan).fundContest(0);

    vm.stopPrank();

    return Pot(contest);
}

function test_PoC_claimCutafterDeadline() public mintAndApproveTokens {
```

```
    Pot pot = _createFundedPot(10, 20, 30, 60);

    // player2 claims in time
    vm.prank(player2);
    pot.claimCut();

    // the 90 days are over
    vm.warp(block.timestamp + 91 days);

    // player1 did not claim in time, but can still claim
    uint256 player1BeforeClaim = weth.balanceOf(player1);
    vm.prank(player1);
    pot.claimCut(); // should revert, but succeeds

    assertEq(weth.balanceOf(player1), player1BeforeClaim + 10);
    assertEq(pot.getRemainingRewards(), 30);

    // the owner closes the contest
    uint256 player1BeforeClose = weth.balanceOf(player1);
    uint256 player2BeforeClose = weth.balanceOf(player2);

    vm.prank(user);
    ContestManager(conMan).closeContest(address(pot));

    uint256 player1ClosingShare = weth.balanceOf(player1) -
    player1BeforeClose;
    uint256 player2ClosingShare = weth.balanceOf(player2) -
    player2BeforeClose;

    console.log("late player1 closing share   :", player1ClosingShare);
    console.log("on-time player2 closing share:", player2ClosingShare);
    // the late player gets the same closing payment as the on-time player
    assertGt(player1ClosingShare, 0);
    assertEq(player1ClosingShare, player2ClosingShare);
}
```

Run from the repository root:

```
forge test --mt test_PoC_claimCutafterDeadline -vv
```

Output:

```
[PASS] test_PoC_claimCutafterDeadline() (gas: 1239315)
Logs:
  ...
  late player1 closing share   : 9
  on-time player2 closing share: 9
```

The test passes because its assertions confirm the vulnerable behavior: the late claim does not revert, and the late player receives the same closing payment as the on-time player.

Recommended Mitigation

Add deadline check before claiming the rewards

```
+   error Pot__ClaimPeriodOver();
    function claimCut() public {
+   if (block.timestamp - i_deployedAt >= 90 days) {
+            revert Pot__ClaimPeriodOver();
+   }
    address player = msg.sender;
   ....
    }
```

[A-02] Closing can repeat payments

Root + Impact

`Pot::closePot` pays the manager and the claimants, but it does not change `remainingRewards` and it does not mark the Pot as closed. So nothing stops the owner from calling `closeContest` for the same Pot again. If the Pot still holds tokens, the second call pays the manager and the claimants again with the same amounts.

Description

A close should happen only once. After it, the Pot should have nothing left to distribute. But `closePot` never reduces `remainingRewards`:

```
function closePot() external onlyOwner {
    ...
    if (remainingRewards > 0) {
        uint256 managerCut = remainingRewards / managerCutPercent;
        i_token.transfer(msg.sender, managerCut);
        uint256 claimantCut = (remainingRewards - managerCut) /
        i_players.length;
        for (uint256 i = 0; i < claimants.length; i++) {
        _transferReward(claimants[i], claimantCut);
       }
       // remainingRewards is never reduced and there is no "closed" flag
    }
}
```

`remainingRewards` stays the same after the first close, so the check `remainingRewards > 0` is still true on the second call and all payments are repeated.

The second call only works if the Pot still holds tokens after the first close. This is the case in these situations:

- some tokens stay in the Pot because of the division problem in `closePot` (reported separately), or
- the contest was funded more than once, because `fundContest` has no "already funded" check.

If the Pot is empty after the first close, the second call fails because there are no tokens to send. That is not a protection by the code: it only works by accident.

Risk

Likelihood: Low

- Only the owner can call `closeContest`, so the owner has to call it again for the same contest (for example by mistake or a double click).
- Nothing in the code warns or stops the owner.
- The Pot must still hold tokens after the first close.

Impact: Medium

- Example from the PoC: 3 players, only player1 claims. The first close pays the manager 5 and player1 15. The second close pays the manager 5 and player1 15 again.
- So the manager gets 10 instead of 5, and player1 gets 15 more than the protocol intends.
- This money comes out of the tokens that are still in the Pot. Players who did not claim yet lose part of that money.
- Only the money of this one contest is affected.

Proof of Concept

Add the following functions inside the existing `TestMyCut` contract in `test/TestMyCut.t.sol`.

```
address player3 = makeAddr("player3");

 function _createFundedPot(
    uint256 r1,
    uint256 r2,
    uint256 r3,
    uint256 total
    ) internal returns (Pot) {
    address[] memory p = new address[](3);
    p[0] = player1;
    p[1] = player2;
    p[2] = player3;

    uint256[] memory r = new uint256[](3);
    r[0] = r1;
    r[1] = r2;
    r[2] = r3;

    vm.startPrank(user);
    contest = ContestManager(conMan).createContest(
    p,
    r,
    IERC20(ERC20Mock(weth)),
```

```
        total
        );

        ContestManager(conMan).fundContest(0);

        vm.stopPrank();

        return Pot(contest);
    }

    function test_PoC_doubleClose_paysTwice() public mintAndApproveTokens {
        Pot pot = _createFundedPot(10, 20, 30, 60);

        vm.prank(player1);
        pot.claimCut();

        vm.warp(block.timestamp + 91 days);

        uint256 remainingBeforeClose = pot.getRemainingRewards();

        vm.prank(user);
        ContestManager(conMan).closeContest(address(pot)); // first close

        assertEq(weth.balanceOf(conMan), 5);
        assertEq(weth.balanceOf(player1), 25);
        assertEq(weth.balanceOf(address(pot)), 30);

        uint256 remainingAfterFirstClose = pot.getRemainingRewards();
        uint256 conManAfterFirstClose = weth.balanceOf(conMan);
        uint256 player1AfterFirstClose = weth.balanceOf(player1);

        vm.prank(user);
        ContestManager(conMan).closeContest(address(pot)); // second close

        console.log("Remaining before close:", remainingBeforeClose);
        console.log(
        "Remaining after first close", remainingAfterFirstClose);
        console.log("Remaining after second close", pot.getRemainingRewards());
        console.log("Manager paid again:", weth.balanceOf(conMan) -
    conManAfterFirstClose);
        console.log("Player1 paid again:", weth.balanceOf(player1) -
    player1AfterFirstClose);

        assertEq(remainingAfterFirstClose, remainingBeforeClose);
        assertEq(pot.getRemainingRewards(), remainingAfterFirstClose); // never
    decreased
        assertGt(weth.balanceOf(conMan), conManAfterFirstClose);
        assertGt(weth.balanceOf(player1), player1AfterFirstClose);
    }
```

Run from the repository root:

```
forge test --mt test_PoC_doubleClose_paysTwice -vv
```

Output:

```
[PASS] test_PoC_doubleClose_paysTwice() (gas: 1228699)
Logs:
  ...
  Remaining before close: 50
  Remaining after first close 50
  Remaining after second close 50
  Manager paid again: 5
  Player1 paid again: 15
```

Recommended Mitigation

Update the state before sending any tokens, so a second call has nothing left to pay:

```
function closePot() external onlyOwner  {
    ...
    if (remainingRewards > 0) {
        uint256 managerCut = remainingRewards / managerCutPercent;
+       uint256 distributable = remainingRewards - managerCut;
+       remainingRewards = 0; // update state before the transfers
        i_token.transfer(msg.sender, managerCut);

-     uint256 claimantCut = (remainingRewards - managerCut) /
i_players.length;
+     uint256 claimantCut = distributable / i_players.length;
        ...
        }
   ....
    }
```

(The division by `i_players.length` is a separate issue and is fixed in its own report.) A simple `closed` flag that `closePot` checks at the start would also work.

[A-03] A false transfer result consumes a claim without payment

Root + Impact

`Pot::_transferReward` calls `i_token.transfer(...)` and ignores the returned `bool`. The EIP-20 standard allows a token to return `false` instead of reverting when a transfer fails, and says that callers must handle it. If this happens in `claimCut`, the Pot has already set the player's reward to 0, reduced `remainingRewards` and added the player to `claimants`, but no tokens were sent. The player cannot claim again, so the reward is lost.

Description

`claimCut` updates the state first and then calls `_transferReward`, which does not check the result:

```
function claimCut() public {
    address player = msg.sender;
    uint256 reward = playersToRewards[player];
    if (reward <= 0) {
        revert Pot__RewardNotFound();
    }
    playersToRewards[player] = 0;
    remainingRewards -= reward;
    claimants.push(player);
 _transferReward(player, reward);
}

function _transferReward(address player, uint256 reward) internal {
 i_token.transfer(player, reward); // return value is ignored
}
```

If `transfer` returns `false` (for example a token that blocks some addresses or is paused, and returns `false` instead of reverting), `claimCut` does not revert. The player's reward is already set to 0, so the player cannot try again.

All token calls in the protocol ignore the returned `bool`:

- `Pot::_transferReward` -> `i_token.transfer(player, reward)` (used by `claimCut` and `closePot`)
- `Pot::closePot` -> `i_token.transfer(msg.sender, managerCut)`
- `ContestManager::fundContest` -> `token.transferFrom(msg.sender, address(pot), totalRewards)`

In `fundContest`, a `false` result means the owner can believe the Pot is funded when no tokens arrived.

The README says "Standard ERC20 Tokens Only". A token that returns `false` on failure is still allowed by EIP-20, so the code must handle it.

Risk

Likelihood: Low

- It needs a token that returns `false` instead of reverting when a transfer fails.
- The token is chosen by the owner, and players cannot influence it.

Impact: Medium

- The affected player loses the whole reward: the balance is unchanged, `checkCut` is 0 and the claim cannot be repeated.
- The Pot keeps more tokens than `remainingRewards` says, so these tokens belong to nobody.
- Only the money of this one contest is affected.

Proof of Concept

Add the following inside the existing `TestMyCut` contract in `test/TestMyCut.t.sol`. It uses the existing setup, imports and the `mintAndApproveTokens` modifier.

```
address player3 = makeAddr("player3");

function _createFundedPot(
    uint256 r1,
    uint256 r2,
    uint256 r3,
    uint256 total
) internal returns (Pot) {
    address[] memory p = new address[](3);
    p[0] = player1;
    p[1] = player2;
    p[2] = player3;

    uint256[] memory r = new uint256[](3);
    r[0] = r1;
    r[1] = r2;
    r[2] = r3;

    vm.startPrank(user);
    contest = ContestManager(conMan).createContest(
        p,
        r,
        IERC20(ERC20Mock(weth)),
        total
    );
    ContestManager(conMan).fundContest(0);
    vm.stopPrank();

    return Pot(contest);
}

function test_PoC_returnFalse_claimIsLost() public mintAndApproveTokens {
    Pot pot = _createFundedPot(10, 20, 30, 60);

    // the token returns false for this transfer instead of reverting
    vm.mockCall(
        address(weth),
        abi.encodeWithSelector(IERC20.transfer.selector, player1,
uint256(10)),
        abi.encode(false)
    );

    vm.prank(player1);
    pot.claimCut(); // does not revert

    console.log("player1 balance       :", weth.balanceOf(player1));
    console.log("player1 reward left   :", pot.checkCut(player1));
```

```
    console.log("remainingRewards      :", pot.getRemainingRewards());
    console.log("tokens held by the Pot:", weth.balanceOf(address(pot)));

    assertEq(weth.balanceOf(player1), 0);   // no tokens received
    assertEq(pot.checkCut(player1), 0);     // reward is gone, claim cannot
be repeated
    assertGt(weth.balanceOf(address(pot)), pot.getRemainingRewards()); //
60 > 50
}
```

Run from the repository root:

```
forge test --mt test_PoC_returnFalse_claimIsLost -vv
```

Output:

```
[PASS] test_PoC_returnFalse_claimIsLost() (gas: 1146371)
Logs:
  ...
  player1 balance       : 0
  player1 reward left   : 0
  remainingRewards      : 50
  tokens held by the Pot: 60
```

`vm.mockCall` makes `transfer(player1, 10)` return `false`, the same as a token that returns `false` for a failed transfer.

Recommended Mitigation

Use OpenZeppelin's `SafeERC20` for every token call. It reverts when a transfer fails or returns `false`, so the whole `claimCut` reverts and the player's reward stays claimable:

```
+ import {SafeERC20} from
"@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

  contract Pot is Ownable(msg.sender) {
+     using SafeERC20 for IERC20;
      ...
      function _transferReward(address player, uint256 reward) internal {
-         i_token.transfer(player, reward);
+         i_token.safeTransfer(player, reward);
      }
```

The PoC covers claimCut. The other calls listed in the Description ignore the returned value in the same way and need the same fix.

[A-04] Repeated funding can leave extra tokens locked

Root + Impact

`fundContest` transfers `totalRewards` tokens to the Pot every time it is called, and nothing records that the Pot was already funded. If the owner calls it twice for the same contest (for example after a retry or a double click), the Pot holds twice the promised amount. The Pot only pays out based on `remainingRewards`, so the extra tokens can never be paid out and stay locked.

Description

```
function fundContest(uint256 index) public onlyOwner {
    Pot pot = Pot(contests[index]);
    IERC20 token = pot.getToken();
    uint256 totalRewards = contestToTotalRewards[address(pot)];

    if (token.balanceOf(msg.sender) < totalRewards) {
        revert ContestManager__InsufficientFunds();
    }

@>  token.transferFrom(msg.sender, address(pot), totalRewards); // no check
if the Pot is already funded
}
```

`claimCut` and `closePot` pay out only from `remainingRewards`, which is set once to `totalRewards` in the constructor. Neither `Pot` nor `ContestManager` has a function that can send the extra tokens back.

Risk

Likelihood: Low

- Only the owner can call `fundContest`, so the owner has to fund the same contest twice by mistake.
- Nothing in the code warns about it.

Impact: Medium

- An amount equal to `totalRewards` (60 tokens in the PoC) is locked in the Pot.
- Only the owner's own tokens for this one contest are affected.

Proof of Concept

Add the following inside the existing `TestMyCut` contract in `test/TestMyCut.t.sol`. It uses the existing setup, imports and the `mintAndApproveTokens` modifier.

```
address player3 = makeAddr("player3");

function _createFundedPot(
    uint256 r1,
    uint256 r2,
```

```
        uint256 r3,
        uint256 total
    ) internal returns (Pot) {
        address[] memory p = new address[](3);
        p[0] = player1;
        p[1] = player2;
        p[2] = player3;

        uint256[] memory r = new uint256[](3);
        r[0] = r1;
        r[1] = r2;
        r[2] = r3;

        vm.startPrank(user);
        contest = ContestManager(conMan).createContest(
            p,
            r,
            IERC20(ERC20Mock(weth)),
            total
        );
        ContestManager(conMan).fundContest(0);
        vm.stopPrank();

        return Pot(contest);
    }

    function test_PoC_fundContestTwice_tokensLocked() public
    mintAndApproveTokens {
        Pot pot = _createFundedPot(10, 20, 30, 60); // funded once inside the
    helper

        // the owner funds the same contest again
        vm.prank(user);
        ContestManager(conMan).fundContest(0);

        console.log("tokens in the Pot:", weth.balanceOf(address(pot)));
        console.log("remainingRewards :", pot.getRemainingRewards());

        // all players claim in time
        vm.prank(player1);
        pot.claimCut();
        vm.prank(player2);
        pot.claimCut();
        vm.prank(player3);
        pot.claimCut();

        vm.warp(block.timestamp + 91 days);
        vm.prank(user);
        ContestManager(conMan).closeContest(address(pot));

        console.log("tokens left in the Pot after close:",
    weth.balanceOf(address(pot)));

        assertEq(pot.getRemainingRewards(), 0);
```

```
    assertEq(weth.balanceOf(address(pot)), 60); // the second funding is
stuck
}
```

Run from the repository root:

```
forge test --mt test_PoC_fundContestTwice_tokensLocked -vv
```

Output:

```
[PASS] test_PoC_fundContestTwice_tokensLocked() (gas: 1219107)
...
  tokens in the Pot: 120
  remainingRewards : 60
  tokens left in the Pot after close: 60
```

Recommended Mitigation

Remember that a Pot was funded and refuse a second funding. The Pot then always holds exactly the promised amount:

```
+ error ContestManager__AlreadyFunded();
+ mapping(address => bool) public isFunded;

  function fundContest(uint256 index) public onlyOwner {
      Pot pot = Pot(contests[index]);
+     if (isFunded[address(pot)]) revert ContestManager__AlreadyFunded();
+     isFunded[address(pot)] = true;
      ...
  }
```

[A-05] One reverting claimant transfer blocks closing

Root + Impact

`closePot` sends tokens to every claimant in one loop. If the transfer to a single claimant reverts (for example, the token blocks that address after it has claimed), the whole `closePot` call reverts. The manager's cut and the payments to all other claimants are then blocked for as long as that address stays blocked, and the claimants list cannot be changed.

Description

```
function closePot() external onlyOwner {
    ...
```

```
    if (remainingRewards > 0) {
        uint256 managerCut = remainingRewards / managerCutPercent;
        i_token.transfer(msg.sender, managerCut);

        uint256 claimantCut = (remainingRewards - managerCut) /
i_players.length;
@>      for (uint256 i = 0; i < claimants.length; i++) {
@>          _transferReward(claimants[i], claimantCut);
        }
    }
}
```

All payments happen in one transaction. A revert on one transfer undoes all of them, including the manager's cut. There is no way to remove a claimant from `claimants` or to skip a failing payment.

Some widely used ERC-20 tokens revert when the sender or the receiver is on a block list. Some ERC-20 tokens include a blocklist and revert transfers to blocked addresses. This scenario depends on the protocol supporting such tokens.

Risk

Likelihood: Low

The token must support blocking recipients, and a claimant must become blocked after successfully claiming their initial reward.

Impact: Medium

A single blocked claimant prevents the manager and other claimants from receiving their closing payments. Closing remains blocked while the token continues rejecting that transfer.

Proof of Concept

Add the following inside the existing `TestMyCut` contract in `test/TestMyCut.t.sol`. It uses the existing setup, imports and the `mintAndApproveTokens` modifier.

```
address player3 = makeAddr("player3");

function _createFundedPot(
    uint256 r1,
    uint256 r2,
    uint256 r3,
    uint256 total
) internal returns (Pot) {
    address[] memory p = new address[](3);
    p[0] = player1;
    p[1] = player2;
    p[2] = player3;

    uint256[] memory r = new uint256[](3);
    r[0] = r1;
```

```
        r[1] = r2;
        r[2] = r3;

        vm.startPrank(user);
        contest = ContestManager(conMan).createContest(
            p,
            r,
            IERC20(ERC20Mock(weth)),
            total
        );
        ContestManager(conMan).fundContest(0);
        vm.stopPrank();

        return Pot(contest);
    }

    function test_PoC_blockedClaimant_blocksClosePot()
        public
        mintAndApproveTokens
    {
        Pot pot = _createFundedPot(10, 20, 30, 60);

        // Two players claim normally and enter the claimants list.
        vm.prank(player1);
        pot.claimCut();

        vm.prank(player2);
        pot.claimCut();

        vm.warp(block.timestamp + 91 days);

        // Simulate the token blocking Player 2 after the initial claim.
        bytes memory blockedError =
            abi.encodeWithSignature("Error(string)", "BLOCKED");

        vm.mockCallRevert(
            address(weth),
            abi.encodeWithSelector(
                IERC20.transfer.selector,
                player2,
                uint256(9)
            ),
            blockedError
        );

        // Player 2's failed transfer undoes the entire closing transaction.
        vm.prank(user);
        vm.expectRevert(blockedError);
        ContestManager(conMan).closeContest(address(pot));

        console.log("Player 1 balance:", weth.balanceOf(player1));
        console.log("Player 2 balance:", weth.balanceOf(player2));
        console.log("Pot balance:", weth.balanceOf(address(pot)));
        console.log("ContestManager balance:", weth.balanceOf(conMan));
```

```
    assertEq(weth.balanceOf(player1), 10);
    assertEq(weth.balanceOf(player2), 20);
    assertEq(weth.balanceOf(address(pot)), 30);
    assertEq(weth.balanceOf(conMan), 0);
    assertEq(pot.getRemainingRewards(), 30);

    // Remove the simulated block and confirm that closing succeeds.
    vm.clearMockedCalls();

    vm.prank(user);
    ContestManager(conMan).closeContest(address(pot));

    assertEq(weth.balanceOf(player1), 19);
    assertEq(weth.balanceOf(player2), 29);
    assertEq(weth.balanceOf(conMan), 3);
    assertEq(weth.balanceOf(address(pot)), 9);
}
```

Run from the repository root:

```
forge test --mt test_PoC_blockedClaimant_blocksClosePot -vv
```

Output:

```
[PASS] test_PoC_blockedClaimant_blocksClosePot() (gas: 1293249)
Logs:
  ...
  Player 1 balance: 10
  Player 2 balance: 20
  Pot balance: 30
  ContestManager balance: 0
```

The mock demonstrates the effect of a reverting transfer. It does not establish that blocklist-enabled tokens are supported by the contest.

Recommended Mitigation

Store closing payments instead of transferring them inside the loop. Let each claimant withdraw separately, so one failed transfer does not block everyone else.

```
+ import {SafeERC20} from "lib/openzeppelin-
contracts/contracts/token/ERC20/utils/SafeERC20.sol";

  contract Pot is Ownable(msg.sender) {
+     using SafeERC20 for IERC20;
+
```

```
+     mapping(address => uint256) public closingPayments;

      function closePot() external onlyOwner {
          ...
          for (uint256 i = 0; i < claimants.length; i++) {
-             _transferReward(claimants[i], claimantCut);
+             closingPayments[claimants[i]] += claimantCut;
          }
      }

+     function claimClosingPayment() external {
+         uint256 payment = closingPayments[msg.sender];
+         require(payment > 0, "No closing payment");
+
+         closingPayments[msg.sender] = 0;
+         i_token.safeTransfer(msg.sender, payment);
+     }
  }
```

If a withdrawal reverts, its state changes are rolled back, allowing that claimant to retry later. Other claimants can still withdraw their payments.

[A-06] Extra reward entries are silently ignored

Root + Impact

The constructor loops over players without checking that both arrays have the same length. If rewards is longer, its extra entries are ignored. Meanwhile, remainingRewards is set to the separately supplied totalRewards, which can exceed the rewards assigned to players.

Description

The constructor assigns rewards using this loop:

```
for (uint256 i = 0; i < i_players.length; i++) {
    playersToRewards[i_players[i]] = i_rewards[i];
}
```

There is no check that players.length == rewards.length.

For example, the owner supplies two players, rewards [1, 2, 3], and totalRewards = 6. Only rewards 1 and 2 are assigned. The third reward is ignored, while remainingRewards is set to 6.

Checking that sum(rewards) == totalRewards would not catch this example: both values are 6.

If rewards is shorter than players, deployment reverts with an array out-of-bounds error. The silent omission occurs when rewards is longer.

Risk

Likelihood: Low

The trusted owner must accidentally supply arrays of different lengths. An ordinary player cannot create this configuration.

Impact: Low

- Some of the funded tokens are not assigned to any player. The constructor does not report this, so the owner does not notice the mistake.
- At closing, part of these tokens may be paid to the players who claimed, but the player the owner forgot to add still gets nothing.
- The closing payments are also affected by the separate division issue (reported separately).

Proof of Concept

Add the following inside the existing `TestMyCut` contract in `test/TestMyCut.t.sol`. It uses the existing setup, imports and the `mintAndApproveTokens` modifier.

```
function test_PoC_claimableSumLessThanRemaining() public
mintAndApproveTokens {
    address[] memory p = new address[](2);
    p[0] = player1;
    p[1] = player2;

    uint256[] memory r = new uint256[](3); // one reward more than players
    r[0] = 1;
    r[1] = 2;
    r[2] = 3;

    vm.startPrank(user);
    contest = ContestManager(conMan).createContest(
        p,
        r,
        IERC20(ERC20Mock(weth)),
        6
    );
    ContestManager(conMan).fundContest(0);
    vm.stopPrank();

    Pot pot = Pot(contest);
    uint256 claimable = pot.checkCut(player1) + pot.checkCut(player2);

    console.log("players can claim in total:", claimable);
    console.log("remainingRewards:", pot.getRemainingRewards());

    assertEq(claimable, 3);
    assertEq(pot.getRemainingRewards(), 6);
    assertLt(claimable, pot.getRemainingRewards());
}
```

Run from the repository root:

```
forge test --mt test_PoC_claimableSumLessThanRemaining -vv
```

Output:

```
[PASS] test_PoC_claimableSumLessThanRemaining() (gas: 1057764)
  ...
  players can claim in total: 3
  remainingRewards: 6
```

Recommended Mitigation

Check the lengths when the Pot is created, so a wrong input fails immediately:

```
+ error Pot__LengthMismatch();

  constructor(address[] memory players, uint256[] memory rewards, IERC20
token, uint256 totalRewards) {
+     if (players.length != rewards.length) revert Pot__LengthMismatch();
      ...
  }
```

Why not matched

The AI feedback says mismatched array lengths are absent from the answer key. Its Issue 7 concerns oversized arrays and an inaccurate `totalRewards`, which is a different root cause.

Quality feedback: the test and direct length check were described positively. It requested precise accounting and removal of the unsupported suggestion that a particular intended player was omitted. This example proves an unmatched reward entry, not the identity or existence of an omitted player.

Evidence Notes

- PoC amounts are token base units.
- Passing PoCs demonstrate original behavior, not the safety of a patched implementation.
- The two token-failure tests use mocks; they do not prove support for every token feature.
- Match labels and quoted rejection reasons come from the supplied AI results and feedback. Quality suggestions are separate from the stated reason for not matching.
- This report does not claim to contain every issue in the challenge.